Latest updates: https://dl.acm.org/doi/10.1145/3284360

PDF Download
3284360.pdf
06 April 2026
Total Citations: 21
Total Downloads: 981

Published: 30 January 2019
Accepted: 01 September 2018
Revised: 01 September 2018
Received: 01 April 2017

Citation in BibTeX format

RESEARCH-ARTICLE

SimBa: An Efficient Tool for Approximating Rips-filtration Persistence via Simplicial Batch Collapse

TAMAL K. DEY, The Ohio State University, Columbus, OH, United States

DAYU SHI, The Ohio State University, Columbus, OH, United States

YUSU WANG, The Ohio State University, Columbus, OH, United States

Open Access Support provided by:

The Ohio State University

Journal of Experimental Algorithmics (JEA), Volume 24 (December 2019)
https://doi.org/10.1145/3284360
EISSN: 1084-6654

SimBa: An Efficient Tool for Approximating Rips-filtration Persistence via Simplicial Batch Collapse

TAMAL K. DEY, DAYU SHI, and YUSU WANG, The Ohio State University

In topological data analysis, a point cloud data P extracted from a metric space is often analyzed by computing the persistence diagram or barcodes of a sequence of Rips complexes built on P indexed by a scale parameter. Unfortunately, even for input of moderate size, the size of the Rips complex may become prohibitively large as the scale parameter increases. Starting with the *Sparse Rips filtration* introduced by Sheehy, some existing methods aim to reduce the size of the complex to improve time efficiency as well. However, as we demonstrate, existing approaches still fall short of scaling well, especially for high-dimensional data. In this article, we investigate the advantages and limitations of existing approaches. Based on insights gained from the experiments, we propose an efficient new algorithm, called *SimBa*, for approximating the persistent homology of Rips filtrations with quality guarantees. Our new algorithm leverages a batch-collapse strategy as well as a new Sparse Rips-like filtration. We experiment on a variety of low- and high-dimensional datasets. We show that our strategy presents a significant size reduction and that our algorithm for approximating Rips filtration persistence is an order of magnitude faster than existing methods in practice.

CCS Concepts: • **Theory of computation → Computational geometry**;

Additional Key Words and Phrases: Topological data analysis, persistent homology, rips filtration, approximation, simplicial maps

ACM Reference format:
Tamal K. Dey, Dayu Shi, and Yusu Wang. 2019. SimBa: An Efficient Tool for Approximating Rips-filtration Persistence via Simplicial Batch Collapse. *J. Exp. Algorithmics* 24, 1, Article 1.5 (January 2019), 16 pages.
https://doi.org/10.1145/3284360

1 INTRODUCTION

Recently, topological ideas and methods have emerged as a new paradigm for analyzing complex data [11, 27]. An important line of work in this direction is the theory and applications of persistent homology. It provides a powerful and flexible framework to inspect data for characterizing and summarizing important features that persist across different scales. Since its introduction [28, 29, 38], there have been many fundamental developments [10, 12, 13, 15, 17, 20–22, 42] both to generalize the framework and to provide theoretical understanding for various aspects of it (such as its stability). These developments help to provide foundation and justification for the practical usage of persistent homology (e.g, see [16, 18, 25, 37, 40]).

This work was supported by NSF grant nos. CCF-1318595 and CCF-1526513.
Authors' addresses: T. K. Dey, D. Shi, and Y. Wang, Department of CSE, The Ohio State University, 2015 Neil Avenue, Columbus, Ohio 43210, USA; emails: tamaldey@cse.ohio-state.edu, shi.384@osu.edu, yusu@cse.ohio-state.edu.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1084-6654/2019/01-ART1.5 $15.00
https://doi.org/10.1145/3284360

A determining factor in applying persistent homology to a broad range of data-analysis problems is the availability of efficient and scalable software. Given the rapidly increasing size of modern data, the "efficiency" necessarily concerns both time and space complexities. The original algorithm to compute persistent homology takes $O(n^3)$ time and $O(n^2)$ space for a filtration involving n number of total simplices [28]. Various practical improvements have been suggested [19, 23]. An early software program widely used for computing persistence is Morozov's Dionysus [26]. Later, Bauer et al. developed the PHAT toolbox [5], based on several efficient matrix reduction strategies (mostly focusing on time efficiency) as described in [4]. A more recently developed library, called GUDHI [41], considers the improvement both in time and space efficiencies. In particular, it uses an efficient data structure, called the simplex tree [8], to encode input simplicial complexes and uses the *compressed annotation matrix* [7] to implement the persistent cohomology algorithm. Dionysus, PHAT, and GUDHI offer efficient software for computing persistence induced by inclusions. For our algorithm, we need persistence induced by more general simplicial maps, for which we use Simpers [1] developed on the basis of the algorithm in [24].

The above results and software cater to general-persistence computations. In practice, often the persistence needs to be computed for a particular filtration called the *Vietoris-Rips* or *Rips filtration*. Given a set of points P embedded in $\mathbb{R}^d$ (or in more general metric spaces), the Rips complex $\mathcal{R}^\alpha(P)$ with radius or scale α is the clique complex induced by the set of edges $\{(p, p') \mid d(p, p') \leq \alpha, p, p' \in P\}$. One is interested in the persistent homology induced by the sequence of Rips complexes $\mathcal{R}^{\alpha_1} \subseteq \mathcal{R}^{\alpha_2} \subseteq \cdots \subseteq \mathcal{R}^{\alpha_m}$ for a growing sequence of radii $\alpha_1 \leq \alpha_2 \leq \cdots \leq \alpha_m$. Intuitively, the Rips complex at a specific scale α approximates the union of radius-α balls around sample points in P. Thus, it captures the structure formed by input points P at different scales.

Unfortunately, even for a modest size of n (in the range of thousands), the size of a Rips complex (as well as the slightly more economical Čech complex) becomes prohibitively large as the radius α increases. In [39], Sheehy proposed an elegant solution for this problem by introducing a *Sparse Rips filtration* to approximate the persistent homology of the Rips filtration for a set of points P. An alternative approach of collapsing input points in batches with an increasing radius α was proposed in [24], which leveraged the persistence algorithm proposed in the same paper for filtrations arising out of simplicial maps.

New Work. Given the importance of the Rips filtration in practice, our goal is to investigate the practical performance of the existing proposed methods, understand their advantages and limitations, and develop an efficient implementation for approximating the persistent homology of Rips filtrations. To this end, we make the following contributions.

1. We investigate the advantages and limitations of three existing methods, two based on Sparse Rips [14, 39], and another on Batch collapse [24]. Specifically, experiments show that while the sparse Rips algorithm by Sheehy [39] has a theoretical guarantee on the size of the filtration and gives good approximation of the persistence diagrams for the Rips filtration in practice, it generates large simplicial complexes even for input of moderate size. This problem becomes more severe as the dimension of the input data increases. The algorithm fails to finish for several high-dimensional datasets of moderate size. See Table 1 for some examples. The Batch-collapse approach is much more space efficient (which leads to time efficiency as well). Nevertheless, we find that its size still becomes prohibitive for high-dimensional data.
2. Based on the insights gained from experimenting with the existing approaches, we propose a new algorithm called *SimBa* that approximates a Rips filtration persistence via simplicial batch collapses. Our algorithm is a modification of the previous batch collapse of Rips filtration proposed in [24]. While, theoretically, the modification may not seem major,

empirically, it reduces the size of the filtration significantly. This leads to a much more efficient approximation of the Rips filtration persistence. Furthermore, we show that this modification maintains a similar approximation guarantee to that of the batch collapse of Rips filtration proposed in [24]. We describe the details of an efficient and practical implementation of SimBa, the software of which has been made publicly available from [2].

Two concepts, homology groups of a simplicial complex and simplicial maps between two complexes, are used throughout this article. We refer the reader to any standard text, such as [36] for details. We denote the p-dimensional homology group of a simplicial complex $\mathcal{K}$ under $\mathbb{Z}_2$ coefficients by $H_p(\mathcal{K})$.

2 RIPS FILTRATION AND ITS APPROXIMATION

Given a set of points $P \subset \mathbb{R}^d$, let $\langle p_0, \ldots, p_s \rangle$ denote the s-dimensional simplex spanned by vertices $p_0, \ldots, p_s \in P$. The Rips complex at scalar α is defined as $\mathcal{R}^\alpha(P) = \{\langle p_0, \ldots, p_s \rangle \mid \|p_i - p_j\| \leq \alpha$, for any $i, j \in [0, s]\}$. Now, consider the following *Rips filtration*:

$$\{\mathcal{R}^\alpha(P)\}_{\alpha>0} \;:=\; \mathcal{R}^{\alpha_1}(P) \hookrightarrow \mathcal{R}^{\alpha_2}(P) \cdots \hookrightarrow \mathcal{R}^{\alpha_n}(P) \cdots . \tag{1}$$

The inclusion maps between consecutive complexes above induce homomorphisms between respective homology groups, giving rise to a so-called *persistence module* for dimension p:

$$H_p(\mathcal{R}^{\alpha_1}(P)) \to H_p(\mathcal{R}^{\alpha_2}(P)) \to \cdots \to H_p(\mathcal{R}^{\alpha_n}(P)) \cdots . \tag{2}$$

If a homology class is created at $\mathcal{R}^{\alpha_i}(P)$ (i.e., does not have a pre-image under homomorphism $H_p(\mathcal{R}^{\alpha_{i-1}}) \to H_p(\mathcal{R}^{\alpha_i})$) and dies entering $\mathcal{R}^{\alpha_j}(P)$ (i.e., its image vanishes under homomorphism $H_p(\mathcal{R}^{\alpha_{j-1}}(P)) \to H_p(\mathcal{R}^{\alpha_j}(P))$, then α_i is its *birth time*, α_j is its *death time*, and the difference $\alpha_j - \alpha_i$ is called the *persistence* of the class. In each dimension, the persistence barcodes capture the persistence of such homology classes by using a horizontal bar with left and right endpoints at α_i and α_j, respectively. These *persistence barcodes* of the above Rips filtration are often the target summary of P and/or of the space P samples, which one wishes to compute in topological data analysis.

The main bottleneck for computing the barcodes of a Rips filtration stems from its size blowup. As the parameter α grows, the Rips complex $\mathcal{R}^\alpha(P)$ can become huge very quickly. To address this blowup in size, Sheehy [39] suggested a novel approach of sparsifying the point set P as one proceeds with increasing α in a way that does not alter the barcodes too much. The idea is to replace the original Rips filtration $\{\mathcal{R}^\alpha(P)\}_{\alpha>0}$ on P with a sequence of smaller complexes $\{\mathcal{S}^\alpha\}_{\alpha>0}$ and show that the two sequences *interleave* at the homology level. Then, appealing to the results of interleaving persistence modules [15], one can show that the barcodes of $\{\mathcal{S}^\alpha\}_{\alpha>0}$ approximate those of $\{\mathcal{R}^\alpha(P)\}_{\alpha>0}$ reasonably. The complexes $\mathcal{S}^\alpha$ are constructed as the *union* of Rips-like complexes built on a sequence of subsets of P rather than the entire set P.

The union allows the complexes in $\{\mathcal{S}^\alpha\}_{\alpha>0}$ to be connected with inclusions and, hence, permits using efficient algorithms and software designed for inclusion-induced persistence. However, the size of $\mathcal{S}^\alpha$ may still be large due to the union operation. An alternative is to avoid the union operation but allow deletion or collapse of vertices (and simplices) at a larger-scale α [14, 39], resulting in a sequence of Rips-like complexes connected with simplicial maps instead of inclusions. This approach, which we refer to as *Sparse Rips with collapse*, achieves only moderate improvements in size reduction, however. We find that much more aggressive size reduction can be achieved by considering the collapse in a batched fashion that gives rise to the approach of *Batch-collapsed Rips* [24].

Finally, building on the batch-collapse idea, we propose a new approach, called *SimBa*, which significantly reduces the size of Rips-like complexes and their computations. This is achieved primarily by replacing inter-point distances with *set distances* while computing the complexes. We

show that this approach still provably approximates the barcodes of the original Rips filtration in sequence (1).

In what follows, we provide more details about each existing method along with its performance in practice, which explains the motivation behind the new tool SimBa.

2.1 Sparse Rips Filtration (Inclusions)

Let P be a set of points in a metric space $(\mathcal{M}, d)$. A *greedy permutation* $\{p_1, \ldots, p_n\}$ of P is defined recursively as follows: Let $p_1 \in P$ be any point and define p_i recursively as $p_i = \text{argmax}_{p \in P \setminus P_{i-1}} d(p, P_{i-1})$, where $P_{i-1} = \{p_1, \ldots, p_{i-1}\}$. This gives rise to a nested sequence of subsets $P_1 \subset P_2 \subset \cdots P_n = P$. Furthermore, each subset P_i is locally dense and uniform (net) in the following sense. Define the insertion radius λ_{p_i} of a point p_i as $\lambda_{p_i} = d(p_i, P_{i-1})$. Each P_i is a λ_{p_i}-net of P, meaning that $d(p, P_i) \leq \lambda_{p_i}$ for every $p \in P$ and $d(p, q) \geq \lambda_{p_i}$ for every distinct pair $p, q \in P_i$. These nets can be extended to a single-parameter family of nets as $\{N_\gamma\}$, where $N_\gamma = \{p \in P | \lambda_p > \gamma\}$ is a γ-net of P.

Using the idea of Sheehy [39], Buchet et al. [9] define a Rips-like filtration using the above specific nets and assigning weights to points whose geometric interpretation is explained in [14]. Each point $p \in P$ is associated with a weight $w_p(\alpha)$ at scale α as

$$w_p(\alpha) = \begin{cases} 0 & \text{if } \alpha \leq \frac{\lambda_p}{\varepsilon} \\ \alpha - \frac{\lambda_p}{\varepsilon} & \text{if } \frac{\lambda_p}{\varepsilon} < \alpha \leq \frac{\lambda_p}{\varepsilon(1-\varepsilon)} \\ \varepsilon\alpha & \text{if } \frac{\lambda_p}{\varepsilon(1-\varepsilon)} \leq \alpha, \end{cases}$$

where $0 < \varepsilon < 1$ is an input constant that controls the sparsity of the filtration. Then, the perturbed distance between pairs of points is defined as

$$\hat{d}_\alpha(p, q) = d(p, q) + w_p(\alpha) + w_q(\alpha).$$

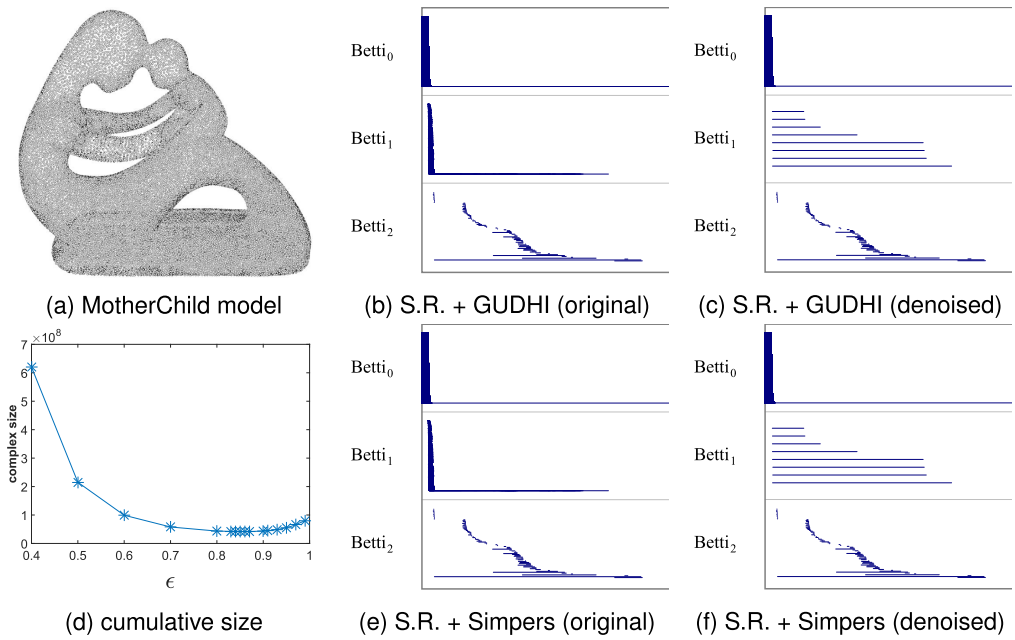
Using the perturbed distance $\hat{d}_\alpha$, the Sparse Rips complex at scale α is defined as

$$Q^\alpha = \{\sigma \subset N_{\varepsilon(1-\varepsilon)\alpha} \mid \forall p, q \in \sigma,\ \hat{d}_\alpha(p, q) \leq 2\alpha\}.$$

The sequence of $\{Q^\alpha\}_{\alpha>0}$ does not form a nested sequence of spaces because the vertex set of each Q^α comes from the net $N_{\varepsilon(1-\varepsilon)\alpha}$ and may decrease as α increases. However, one can take $\mathcal{S}^\alpha = \bigcup_{\alpha' \leq \alpha} Q^{\alpha'}$ and build a natural filtration $\{\mathcal{S}^\alpha \hookrightarrow \mathcal{S}^{\alpha'}\}_{\alpha<\alpha'}$ connected by inclusions. It is shown that the persistence barcodes of the filtration $\{\mathcal{S}^\alpha\}_{\alpha>0}$ approximate those of the Rips filtration $\{\mathcal{R}^\alpha\}_{\alpha>0}$ [39].

We use the code from [9] to compute this Sparse Rips filtration $\{\mathcal{S}^\alpha\}_{\alpha>0}$. We then use GUDHI [41] to compute its persistent barcodes, as GUDHI has the state-of-the-art performance for handling large complexes due to a compression technique [7] for inclusion-based filtrations.

As a common test case to illustrate the performance of various existing methods, we use a 3-dimensional point set sampled from a surface model called MotherChild; see Figure 1(a). We choose this model because the ground truth is available and because existing methods have trouble (to different degrees) handling high-dimensional data. The size of the point set is 23, 075. For indicating memory consumption, we refer to *cumulative size*, which is the total number of simplices arising in the filtration, and to *maximum size*, which is the maximum over all complexes arising in the filtration. For Sparse Rips filtrations, the two sizes coincide at the last complex owing to inclusions. Figure 1(d) shows the cumulative size with different Sparse Rips parameter ε. It is minimum when ε is between 0.8 and 0.9. Thus, we choose $\varepsilon = 0.8$ to achieve the best performance while observing that the approximation quality does not suffer much, as predicted by the theory.



(a) MotherChild model (b) S.R. + GUDHI (original) (c) S.R. + GUDHI (denoised)

(d) cumulative size (e) S.R. + Simpers (original) (f) S.R. + Simpers (denoised)

Fig. 1. MotherChild surface model and its persistence barcodes computed by Sparse Rips (S.R.)–based approaches. Since the surface has genus 4, its barcodes contain long bars: 1 for H_0, 8 for H_1, and 1 for H_2. The minimum cumulative size for complexes, which is about 43 million, is achieved around $\varepsilon = 0.8$.

The original persistence barcodes are shown in Figure 1(b). Since it becomes hard to see the main (long) bars in the presence of all of the spurious ones creating excessive overlaps, we remove all short bars whose ratio between death and birth time is smaller than a threshold for the 1-dimensional homology group H_1. The bars for H_0 and H_2 are not denoised. Unless specified otherwise, all barcodes are denoised in the same way. The denoised barcodes are shown in Figure 1(c), one for H_0, four short and four long for H_1 (MotherChild has genus 4), and one for H_2. The cumulative size of the Sparse Rips complexes in the filtration is 43.5 million and the total time cost is about 350s.

2.2 Sparse Rips with Collapse

The persistence barcodes for the inclusion-based filtration $\{\mathcal{S}^\alpha\}_{\alpha>0}$ are the same as the barcodes of the filtration $\{Q^\alpha\}_{\alpha>0}$ connected by simplicial maps $Q^\alpha \to Q^{\alpha'}$ for $\alpha < \alpha'$. Specifically, these simplicial maps originate from vertex collapses defined by the following projection map:

$$\mu_\alpha(p) = \begin{cases} p & \text{if } p \in N_{\varepsilon(1-\varepsilon)\alpha} \\ \operatorname*{argmin}_{q \in N_{\varepsilon\alpha}} d(p,q) & \text{otherwise.} \end{cases}$$

For any scale α, the projection μ_α maps the points of P to the net $N_{\varepsilon(1-\varepsilon)\alpha} \supseteq N_{\varepsilon\alpha}$. One can view it as p being deleted at time (scale) $\alpha_p = \frac{\lambda_p}{\varepsilon(1-\varepsilon)}$. We can construct the sequence of Sparse Rips complexes $\{Q^\alpha\}_{\alpha>0}$ connected with simplicial maps induced by insertions and vertex collapses as α increases. Specifically, we delete the vertex p and all of its incident simplices by collapsing it to its projection $\mu_{\alpha_p}(p)$, where $\alpha_p = \frac{\lambda_p}{\varepsilon(1-\varepsilon)}$, when entering complex Q^{α_p}. See [14] for more details.

In this approach, we need to compute the persistence induced with simplicial maps. For this, we use the only available software, Simpers [1], based on the algorithm presented in [24]. Our

experimental results on the MotherChild model of Figure 1(a) with $\varepsilon = 0.8$ are given in Figures 1(e) and 1(f). The barcodes are exactly the same as those in Figures 1(b) and 1(c). The cumulative size of the entire sequence is the same, 43.5 million, because the final complex in $\mathcal{S}^\alpha$ is the union of all complexes in $\{Q^\alpha\}_{\alpha>0}$. However, the maximum size in the sequence is 24.9 million owing to vertex collapses in contrast to the maximum size for $\{\mathcal{S}^\alpha\}_{\alpha>0}$, which equals the cumulative size. The time cost for this approach is 463.7s, which is larger than that for Sparse Rips with GUDHI. Thus, compared to Sparse Rips with GUDHI, this approach has a smaller maximum size owing to collapse but costs more time for computing persistence since Simpers computes persistence over collapses, which are slower operations than inclusions.

While the size of these Sparse Rips complexes is linear in the number of input points, the hidden constant factor depends exponentially on the doubling dimension of the sampled metric space. Empirically, we note that the size is still large and becomes much worse as the dimension of data increases. For example, for the Gesture Phase data in Table 1, which has only $1,747$ points in $\mathbb{R}^{18}$, the cumulative size of the Sparse Rips filtration is 45.6 million, which approaches the limit that GUDHI or Simpers can handle. For other larger datasets, such as Primary Circle or Survivin, the complex reaches a size beyond this limit. Moreover, one has to pre-compute a greedy permutation of the input point set before constructing the Sparse Rips filtration. This computation is usually costly, requiring furthest point computations for which software as efficient as ANN (for nearest neighbors) is not available. This motivates us to consider the batched approach in the next section.

2.3 Batch-Collapsed Rips

For handling large and high-dimensional data, we need a more aggressive sparsification than the Sparse Rips filtration. We consider the Batch-collapsed Rips filtration, which has been proposed previously in [24, Section 6.1].

Given a set of points P, first set $V_0 := P$ and compute the shortest pairwise distance α. We next construct a sequence of vertex sets $V_k, k \in [0, m]$ such that V_{k+1} is an αc^{k+1}-net of V_k, where $c > 1$ is a parameter that controls the rate of the scale increase. Consider the vertex map $\pi_k : V_k \to V_{k+1}$, for $k \in [0, m-1]$, such that for any $v \in V_k$, $\pi_k(v)$ is v's nearest neighbor in V_{k+1}. It can be shown that each vertex map π_k induces a well-defined simplicial map $s_k : \mathcal{R}^{\alpha c^k \frac{3c-1}{c-1}}(V_k) \to \mathcal{R}^{\alpha c^{k+1} \frac{3c-1}{c-1}}(V_{k+1})$. The *Batch-collapsed Rips filtration* is:

$$\mathcal{R}^0(V_0) \xrightarrow{s_0} \mathcal{R}^{\alpha c \frac{3c-1}{c-1}}(V_1) \cdots \xrightarrow{s_{m-1}} \mathcal{R}^{\alpha c^m \frac{3c-1}{c-1}}(V_m). \tag{3}$$

Using the line of proof in [24], one can show that the persistence of this sequence is a $3\log(\frac{2}{c-1} + 3)$ approximation of the persistence diagram of Rips filtration given below.

$$\mathcal{R}^0(V_0) \hookrightarrow \mathcal{R}^{\alpha c}(V_0) \cdots \hookrightarrow \mathcal{R}^{\alpha c^m}(V_0). \tag{4}$$

The blowup in scale by the factor of $\frac{3c-1}{c-1}$ results from the proof, which, in practice, causes some problems. We elaborate this further. To satisfy the approximation guarantee, one has to show that the persistence modules arising from Batch-collapsed Rips in sequence (3) and the standard Rips in sequence (4) interleave. In particular, this requires that we have well-defined simplicial maps from complexes in sequence (3) to those in sequence (4) and vice versa. The multiplicative factor $\frac{3c-1}{c-1}$ is needed to ensure that there is a well-defined simplicial map $\mathcal{R}^{\alpha c^k}(V_0) \to \mathcal{R}^{\alpha c^k \frac{3c-1}{c-1}}(V_k)$, as $\mathcal{R}^{\alpha c^k \frac{3c-1}{c-1}}(V_k)$ has to be sufficiently connected to include all images of the simplices in the domain Rips complex $\mathcal{R}^{\alpha c^k}(V_0)$. The side effect of this is that the Batch-collapsed Rips complex has to be built at a much larger scale than the Rips complex. It ends up with many unnecessary connections and, thus, more simplices in practice. This also causes a trade-off: larger c reduces the over-connection but results in a worse approximation factor, leading to a worse approximation

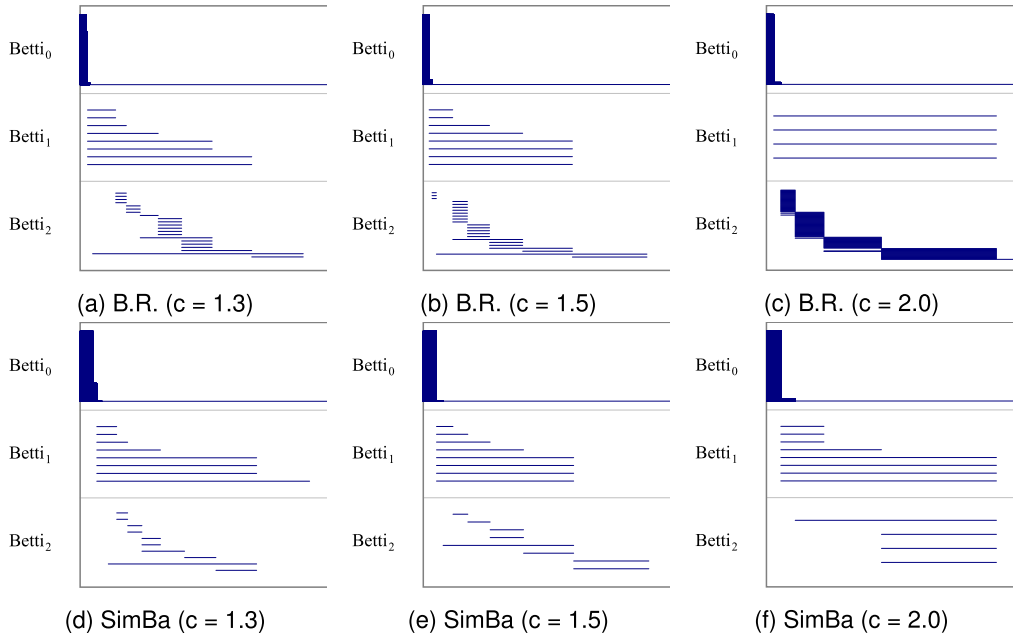


Fig. 2. Persistence barcodes computed by Batch-collapsed Rips plus Simpers (B.R.) and SimBa on the same MotherChild model. B.R. captures main bars for H_1 correctly for smaller values of c, as shown in (a) and (b) and loses some for $c = 2.0$ as shown in (c), while SimBa works for $c = 2.0$.

quality. It is not clear how to set an increase rate that achieves both good approximation quality and efficiency for a specific dataset.

We experimented Batch-collapsed Rips with Simpers on the same MotherChild model. Figures 2(a), 2(b), and 2(c) show the persistence barcodes for different values of c. Observe that smaller values of c give better approximation. The barcode for $c = 1.3$ is the most similar among the three to that of Sparse Rips filtration in Figure 1(b), which is theoretically supposed to be more accurate. On the other hand, when c grows more than 1.8, it starts to lose some main bars in H_1 and noisy bars get longer in H_2. Yet, Figure 3 shows that, as c increases, both complex size and time cost decrease drastically. When $c = 2.0$, it involves less than $216K$ simplices and takes 9.4 seconds while, although $c = 1.3$ gives a more accurate barcode, its size (22.5 million) and time (325s) approach those of the Sparse Rips. This demonstrates the dilemma that Batch-collapsed Rips faces in practice. We address this issue in our new approach, SimBa. In particular, when $c \leq 2$, SimBa performs better than Batch-collapsed Rips for both size and time, as shown in Figure 3, while capturing all main bars correctly, as shown in Figure 2.

3 SIMBA

To tame the over-connection in Batch-collapsed Rips, we replace the sequence in (3) with the sequence below, where the parameter does not incur the extra factor $\frac{3c-1}{c-1}$:

$$\mathcal{B}^0(V_0) \to \mathcal{B}^{\alpha c}(V_1) \to \cdots \mathcal{B}^{\alpha c^m}(V_m) \tag{5}$$

The complexes $\mathcal{B}^{\alpha c^k}(V_k)$ are built on the same vertex sequence $\{V_k\}$ as in Batch-collapsed Rips, but the distances among the vertices of V_k are replaced with a *set distance*, which allows us to avoid the over-connection. For two sets of points (clusters) $A, B \subset P$, we define their set distance as $d(A, B) = \min_{a \in A, b \in B} d(a, b)$. The sets that we consider are the pre-images of the vertices in V_k

under the composition of projections π_is, namely, for a vertex $v \in V_k$, we consider the set

$$B_v^k = \{p \in V_0 \mid \hat{\pi}_k(p) = v\}, \text{ where } \hat{\pi}_k : V_0 \to V_k \text{ is defined as } \hat{\pi}_k = \pi_{k-1} \circ \cdots \circ \pi_0.$$

The complex $\mathcal{B}^{\alpha c^k}(V_k)$ is simply the clique complex induced by the edge set $\{(u, v) \in V_k \mid d(B_u^k, B_v^k) \leq \alpha c^k\}$. Observe that $d(u, v) \geq d(B_u^k, B_v^k)$, which ensures that the normal connection between u and v for a Rips filtration at the respective scale is not missed by considering the set distance while still avoiding the over-connection.

It turns out that each vertex map (nearest neighbor projection) $\pi_k : V_k \to V_{k+1}$ induces a simplicial map $h_k : \mathcal{B}^{\alpha c^k}(V_k) \to \mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$. Instead of recomputing the simplicial complex each time, we generate elementary insertion and collapse operations incrementally for each h_k in three steps: (i) collapse each $v \in V_k \setminus V_{k+1}$ to its image $\pi_k(v)$ in V_{k+1} along with all incident simplices, (ii) insert new edges between two vertices in V_{k+1} if the distance between the two sets they represent are smaller than or equal to the current scale, and (iii) insert all new clique simplices containing new edges generated by (i) and (ii). Each h_k is processed in one batch, starting from a simplicial complex on vertices in V_k and resulting in a simplicial complex on vertices in V_{k+1}. The collapse and insertions of new simplices are exactly what Simpers needs for computing the persistence.

3.1 Implementation Details

The advantage of SimBa (and Batch-collapsed Rips) over Sparse Rips filtrations is mainly due to the batched approach, which requires us to compute δ-nets of a point set for some δ repeatedly. Its advantage over the Batch-collapsed Rips is credited to the use of set distances. These computations require k-nearest neighbor search and fixed radius search, for which efficient libraries such as ANN [35] and Cover tree [6] exist. We take advantage of these available software programs.

To compute a δ-net of a given point set (to obtain V_{k+1} from V_k), we randomly pick an untouched point–say, p–use fixed-radius search to find all points in the ball of radius δ around p, map them to it, and mark them processed. We do this repeatedly until there is no untouched point left. We observe that this sub-sampling procedure can be carried out faster at an early stage when δ is small because those points whose nearest neighbor distances are larger than the current δ can be taken directly into the net–they are all mapped to themselves and no other points are mapped to them. Thus, we maintain a list L of the points ordered by their nearest neighbor distances in increasing order and process them sequentially for δ-net computations. To compute the net points V_{k+1} from V_k, we carry out the full sub-sampling process only on the points in V_k that are already known to have nearest-neighbor distances below δ and the new ones that qualify from L for increased δ. After δ becomes more than the largest nearest-neighbor distance, we convert to the usual net computation.

Next, we describe an efficient implementation of the set distance computation, which, being a basic operation in SimBa, speeds it up significantly. A straightforward implementation requires quadratic time, but we can make it more efficient in practice with the help of the ANN library or Cover tree. We use a hybrid strategy as follows. The sets B_u^k for vertices $u \in V_k$ are maintained by a union-find data structure. As vertices are collapsed while going from V_k to V_{k+1}, the sets of the collapsed vertices are merged to that of the target vertex. At the early stages, when the number of sets (i.e., the size of V_k) is large and the diameter of each set is potentially small, we avoid computing set distances for all pairs. For each processing set B_u^k, we only need to find all of the sets B_v^k whose distances to B_u^k are smaller than the current scale $\alpha' = \alpha c^k$. If so, we add an edge between u and v. To find all of these nearby sets, we can do a fixed-radius search in $V_0 = P$ around each point in B_u^k within α' distance. For each point v returned by the search, we find v in the union-find data structure to identify its image $\hat{\pi}_k(v) \in V_k$. If the representing set of v is different from that of u, we add the edge $\hat{\pi}_k(u)\hat{\pi}_k(v)$.

Later, when α' becomes large, it may not be efficient to continue this fixed-radius search, as the number of candidate points from P may be too large (which can be n in the worst case). Thus, we fall back on pairwise set distance computation. In particular, when the cardinality of V_k becomes lower than a threshold–say, 1/10 of the number of input points–we compute a pairwise set distance matrix (of size $|V_k| \times |V_k|$) among the surviving sets once and then keep updating the matrix with batch collapse thereafter. In particular, note that given sets A, B, and C, the set distance $d(A \cup B, C) = \min\{d(A, C), d(B, C)\}$.

The only thing that remains is to find all new simplices, hence, cliques involving a new edge efficiently. For this, we use the following Algorithm 1, which is a slight modification of the Bron-Kerbosch algorithm. The variables R and P represent vertex sets. In batch collapse, we maintain an underlying graph that contains the current connectivity of the surviving sets. Then, $N(v)$ represents the set of neighbors of vertex v in the graph. For each new edge, we initialize R to contain the two vertices of the edge and P to contain their common neighbors in the current graph. The algorithm reports all cliques recursively in step 4.

ALGORITHM 1: Find All New Clique Simplices Involving a New Edge (a, b)

1: $R := \{a, b\}$
2: $P := N(a) \cap N(b)$
3: **procedure FindCliques**(R, P)
4: Report R as a new clique (simplex)
5: **while** $P \neq \emptyset$ **do**
6: Let $v \in P$ be any vertex
7: **FindCliques**($R \cup \{v\}$, $P \cap N(v)$)
8: $P := P \setminus \{v\}$

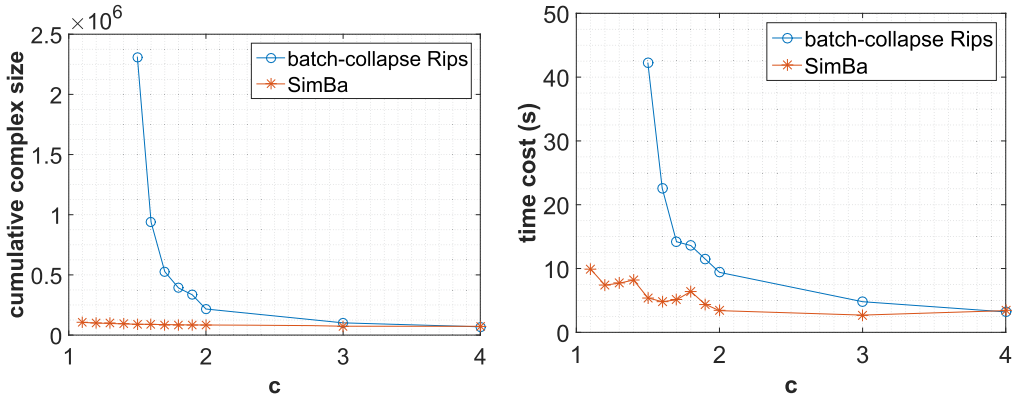
3.2 SimBa on MotherChild Model

We compare SimBa with other approaches on the same MotherChild model. Figures 2(d), 2(e), and 2(f) show the persistence barcodes computed by SimBa with different values of c. We see that SimBa captures all of the main 0-, 1-, and 2-dimensional bars for all values of c in the range from 1.3 to 2.0 as opposed to Batch-collapsed Rips, which fails to capture the main H_1 bars for $c > 1.8$. It tolerates a larger range of c and, thus, is more robust than Batch-collapsed Rips. As expected, larger values of c produce less bars since there are less batches. Thus, in practice, we should choose a smaller c, say, less than 1.5. More importantly, as Figure 3 shows, the size and time for SimBa are also stable against different values of c, all less than $100K$ simplices and 10s, respectively, for $c \leq 2$. These are less than those for Batch-collapsed Rips and significantly less than those for Sparse Rips. In particular, when $c = 1.3$, the maximum size for SimBa is $100K$, similar to when $c = 2$. However, for Batch-collapsed Rips, the maximum size is closer to that of SimBa when $c = 2$, and is 22.5 and 1.4 million when $c = 1.3$ and $c = 1.5$, respectively. This size difference becomes even more prominent for high-dimensional data, as Table 1 shows. Although the approximation quality of SimBa is slightly worse than that of Sparse Rips–based approaches, it does capture all of the main bars and, more importantly, costs significantly less time. This advantage allows SimBa to process much larger high-dimensional datasets that no previous approaches could handle, as we illustrate in Section 5.

4 APPROXIMATION GUARANTEE OF SIMBA

Recall that the simplicial complex $\mathcal{B}^\alpha(V_k)$ appearing in SimBa's filtration is defined as

$$\mathcal{B}^\alpha(V_k) = \{\sigma \subset V_k \mid \forall u, v \in \sigma, d(B_u^k, B_v^k) \leq \alpha\}.$$



(a) cumulative size (b) time cost

Fig. 3. Complex size and time cost comparison between Batch-collapsed Rips and SimBa. SimBa beats Batch-collapsed Rips for both size and time when $c \leq 2$. For $c > 2$, the barcodes of both batch-based approaches become too coarse to be useful in practice.

We prove that the persistence barcodes of SimBa's filtration in sequence (5) approximates those of the Rips filtration in (4) by showing that the persistence modules induced by these two sequences interleave.

First, observe that each vertex map π_k induces a well-defined simplicial map $h_k : \mathcal{B}^{\alpha c^k}(V_k) \to \mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$. For any edge $\{u, v\}$ in $\mathcal{B}^{\alpha c^k}(V_k)$, suppose that $u' = \pi_k(u), v' = \pi_k(v)$; then, $B_u^k \subset B_{u'}^{k+1}$ and $B_v^k \subset B_{v'}^{k+1}$. Thus, we have that $d(B_{u'}^{k+1}, B_{v'}^{k+1}) \leq d(B_u^k, B_v^k) \leq \alpha c^k < \alpha c^{k+1}$. Therefore, $\{u', v'\}$ must be an edge in $\mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$ as well. Since each complex in SimBa's filtration is a clique complex determined by edges, every simplex in $\mathcal{B}^{\alpha c^k}(V_k)$ has a well-defined image in $\mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$. Thus, each h_k is well defined.

Recall that the map $\hat{\pi}_k : V_0 \to V_{k+1}$ is defined as $\hat{\pi}_k(v) = \pi_k \circ \cdots \circ \pi_0(v)$, which tracks the image of any point in $V_0 = P$ during the batch-collapse process. Observe that the vertex map $\hat{\pi}_k$ also induces a simplicial map $\hat{h}_k : \mathcal{R}^{\alpha c^k}(V_0) \to \mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$: specifically, for any edge $(u, v) \in \mathcal{R}^{\alpha c^k}(V_0)$ with $d(u, v) \leq \alpha c^k$, it is easy to see that $d(B^k_{\hat{\pi}_k(u)}, B^k_{\hat{\pi}_k(v)}) \leq d(u, v) \leq \alpha c^k < \alpha c^{k+1}$, implying that $(\hat{\pi}_k(u), \hat{\pi}_k(v))$ is an edge in $\mathcal{B}^{\alpha c^{k+1}}(V_{k+1})$. The key observation is in the following lemma.

Lemma 4.1. *Each triangle in the diagram below commutes at homology level, where i_k and j_k are induced by inclusions, $h_{k,t} := h_{k+t-1} \circ \cdots \circ h_k$, $c > 1$, $t \geq \log_c(\frac{2}{c-1} + 3)$ and $t \in Z$.*

$$\begin{tikzcd}
\mathcal{R}^{\alpha c^k}(V_0) \arrow[r, hook, "i_k"] \arrow[d, "\hat{h}_k"] & \mathcal{R}^{\alpha c^{k+t}}(V_0) \arrow[d, "\hat{h}_{k+t}"] \\
\mathcal{B}^{\alpha c^k}(V_k) \arrow[r, "h_{k,t}"] \arrow[ur, hook, "j_k"] & \mathcal{B}^{\alpha c^{k+t}}(V_{k+t})
\end{tikzcd}$$

Proof. First, we prove that there is, indeed, an inclusion map $j_k : \mathcal{B}^{\alpha c^k}(V_k) \hookrightarrow \mathcal{R}^{\alpha c^{k+t}}(V_0)$. In particular, we show that, for each edge (u, v) in $\mathcal{B}^{\alpha c^k}(V_k)$, it is also an edge in $\mathcal{R}^{\alpha c^{k+t}}(V_0)$. Suppose that the set distance $d(B_u^k, B_v^k)$ is achieved by the closest pair (u_0, v_0) between the two sets, where $u_0 \in B_u^k, v_0 \in B_v^k$. Then, $d(B_u^k, B_v^k) = d(u_0, v_0) \leq \alpha c^k$. Since V_{i+1} is an αc^{i+1}-net of V_i for each $i \in [0, k-1]$, it follows that $d(u, u_0) \leq \alpha c^k \sum_{i=0}^{k-1} \frac{1}{c^i} < \alpha c^k \frac{c}{c-1}$. A similar bound holds for $d(v, v_0)$.

Thus,

$$d(u,v) \le d(u,u_0) + d(v,v_0) + d(u_0,v_0) \le \alpha c^k \left(\frac{2c}{c-1} + 1\right) = \alpha c^k \left(\frac{2}{c-1} + 3\right) \le \alpha c^{k+t}.$$

Hence, u, v is an edge in $\mathcal{R}^{\alpha c^{k+t}}(V_0)$.

Next, observe that the vertex map $\hat{\pi}_{k+t}$ restricted on the set of vertices V_k is exactly the same as the vertex map $\pi_{k,t} := \pi_{k+t-1} \circ \cdots \circ \pi_k$ (this vertex map induces the simplicial map $h_{k,t}$ in the diagram). Namely, for a vertex $u \in V_k \subseteq V_0$, $h_{k,t}(u) = \hat{h}_{k+t}(u)$. Thus, $h_{k,t} = \hat{h}_{k+t} \circ j_k$. Hence, the bottom triangle commutes both at the complex and homology levels.

We now consider the top triangle. Specifically, we prove that the map $j_k \circ \hat{h}_k$ is contiguous to the inclusion map i_k. Since two contiguous maps induce the same homomorphisms at the homology level, the top triangle commutes at the homology level.

Given a simplex $\sigma \in \mathcal{R}^{\alpha c^k}(V_0)$, we need to show that vertices from $i_k(\sigma) \cup j_k \circ \hat{h}_k(\sigma)$ span a simplex in $\mathcal{R}^{\alpha c^{k+t}}(V_0)$. Since both are Rips complexes and i_k and j_k are inclusion maps, we need to prove only that for any two vertices u and v from $\sigma \cup \hat{h}_k(\sigma)$, $d(u,v) \le \alpha c^{k+t}$ (i.e., (u,v) is an edge in $\mathcal{R}^{\alpha c^{k+t}}(V_0)$). If u and v are both from σ or both from $\hat{h}_k(\sigma)$, then $d(u,v) \le \alpha c^{k+t}$ trivially. Otherwise, assume without loss of generality that $v \in \sigma$ and $u \in \hat{h}_k(\sigma)$, where $u = \hat{\pi}_k(u')$ for some $u' \in \sigma$. Since V_{i+1} is an αc^{i+1}-net of V_i for each $i \in [0, k-1]$, it follows that $d(u,u') \le \alpha c^k \sum_{i=0}^{k-1} \frac{1}{c^i} < \alpha c^k \frac{c}{c-1}$. One then has that

$$d(u,v) \le d(u,u') + d(u',v) \le \alpha c^k \frac{c}{c-1} + \alpha c^k = \alpha c^k \frac{2c-1}{c-1} < \alpha c^k \left(\frac{2}{c-1} + 3\right) \le \alpha c^{k+t}.$$

Thus, i_k is contiguous to $j_k \circ \hat{h}_k$ and the lemma follows. □

The above result implies that the persistence modules induced by sequences (5) and (4) are weakly $\log c^t$ interleaved at the log scale. Since $t \ge \log_c(\frac{2}{c-1} + 3)$, we have that $c^t \ge \frac{2}{c-1} + 3$. By Theorem 4.3 of [15], we conclude with the following:

Theorem 4.2. *The persistence diagram of sequence (5) provides a* $3\log(\frac{2}{c-1} + 3)$ *approximation of the persistence diagram of tsequence (4) at the log scale for* $c > 1$.

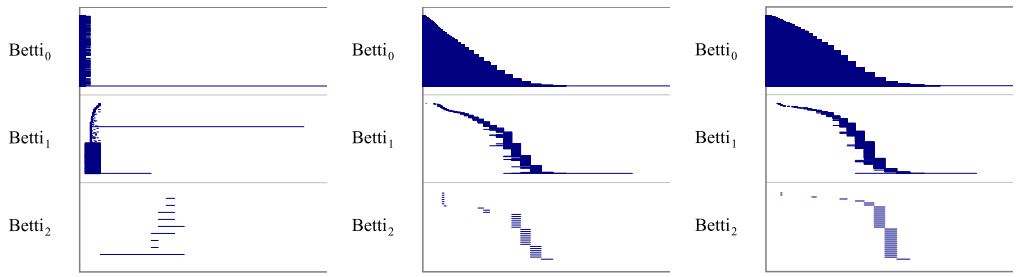
5 EXPERIMENTS

In this section, we report some experimental results of SimBa on large high-dimensional datasets from fields such as machine learning, image processing, and computational biology. In general, we experiment on SimBa in two main settings, both Euclidean space and general metric spaces. For Euclidean space, we use the ANN library [35] to support nearest-neighbor search. It is also convenient to compare performances of different sparsification approaches in this setting. For general metric spaces for which ANN is no longer appropriate, we use Cover tree [6] implementation instead for nearest-neighbor search. Both options are available in our publicly distributed SimBa software [2].

All of the following experiments were run on a 64b Windows machine with a 3.50GHz Intel processor and 16GB RAM. All approaches are implemented in C++. Note that we compute persistence only up to dimension 2 (which means that we build simplicial complexes up to dimension 3).

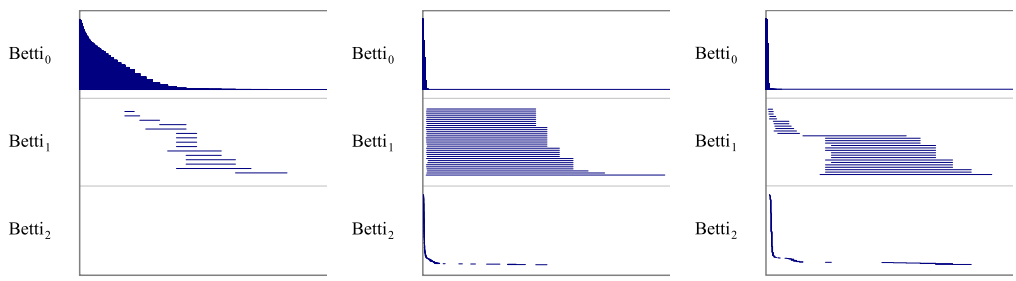
5.1 Data from Euclidean Spaces

For most of the tested datasets, previous approaches are not efficient enough to finish processing. They either ran out of memory ("∞" in size) or ran more than one day ("∞" in time). Table 1 provides the cumulative size and time cost for all four approaches mentioned in this article. For Sparse Rips with GUDHI and Sparse Rips with Simpers, we choose parameter $\varepsilon = 0.8$, which gives



(a) Klein Bottle in $\mathbb{R}^4$ (b) Primary Circle in $\mathbb{R}^{25}$ (c) Primary Circle in $\mathbb{R}^{49}$

Fig. 4. Original persistence barcodes computed by SimBa on datasets with ground truth.



(a) Gesture Phase data in $\mathbb{R}^{18}$ (b) Survivin data in $\mathbb{R}^3$ (c) Survivin data in $\mathbb{R}^{150}$

Fig. 5. Denoised persistence barcodes computed by SimBa on data sets without ground truth.

the best performance while not sacrificing much of the approximation quality. For Batch-collapsed Rips with Simpers, we choose $c = 1.5$, which appears to reach a good trade-off between efficiency and quality. For SimBa, we choose $c = 1.1$, which, in practice, appears to have the best quality. Note that the choice of c does not seem to change the empirical efficiency much, as Figure 3 illustrates.

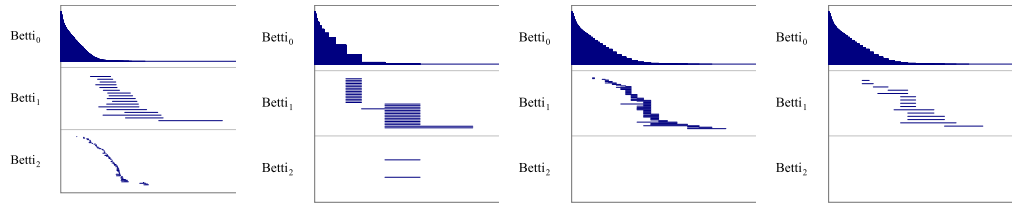
Data with Ground Truth. We first test with two datasets whose ground truth persistence values are known. They help demonstrate that SimBa works properly and efficiently in practice. All persistence barcodes shown in Figure 4 are original and not cleaned up.

We first consider a uniform sample of 22, 500 points from a Klein bottle in $\mathbb{R}^4$ and use SimBa to compute its barcode, which is shown in Figure 4(a). There are two main bars for H_1 and one for H_2, as expected.

Next, we consider the primary circle of natural image data in [3], which has 15,000 points. Each point is a 5×5 or 7×7 image patch, thus considered as a point in $\mathbb{R}^{25}$ or $\mathbb{R}^{49}$. From Figures 4(b) and 4(c), we can see the primary circle bar for H_1 for data both in $\mathbb{R}^{25}$ and $\mathbb{R}^{49}$. All short bars for H_2 persist for only one batch step and, thus, can be regarded as noise.

Data without Ground Truth. Next, we provide some experiments on the datasets whose ground truth persistence values are not known. We used SimBa to compute their persistence and found some relatively long bars that are likely to be features and worth being investigated further by domain experts. The persistence barcodes shown in Figures 5 and 6(d) are denoised for H_1. The rest of the barcodes in Figure 6 are original.

We first take the Gesture Phase Segmentation dataset [34] from the UCI machine-learning repository [32]. This dataset was used in [33]. It comprises features extracted from 7 videos with people gesticulating. Each video is represented by a raw file that contains the positions of hands, wrists,



(a) S.R.+GUDHI (b) B.R.+Simpers (c) SimBa (d) SimBa (denoised)

Fig. 6. Persistence barcodes computed by different approaches on Gesture Phase Segmentation data in $\mathbb{R}^{18}$.

Table 1. Cumulative Size and Time Cost

				S.R.+GUDHI		S.R.+Simpers		B.R.+Simpers		SimBa	
Data	n	D	d	size	time(s)	size	time(s)	size	time(s)	size	time(s)
Mother	23,075	3	2	$43.5 \cdot 10^6$	350	$43.5 \cdot 10^6$	463.7	$2.3 \cdot 10^6$	42.3	104,701	8.8
KlBt	22,500	4	2	$20.9 \cdot 10^6$	205.3	$20.9 \cdot 10^6$	303.5	440049	8	78,064	6.6
PrCi25	15,000	25	?	∞	–	∞	–	–	∞	$4.8 \cdot 10^6$	216
PrCi49	15,000	49	?	∞	–	∞	–	–	∞	$10.2 \cdot 10^6$	585
GePh	1,747	18	?	$45.6 \cdot 10^6$	282.5	$45.6 \cdot 10^6$	432.8	$1.4 \cdot 10^6$	29	7,145	0.83
Sur3	252,996	3	?	∞	–	∞	–	$15.7 \cdot 10^6$	1056.4	915,110	1,079.6
Sur150	252,996	150	?	∞	–	∞	–	–	∞	$3.1 \cdot 10^6$	5,089.7

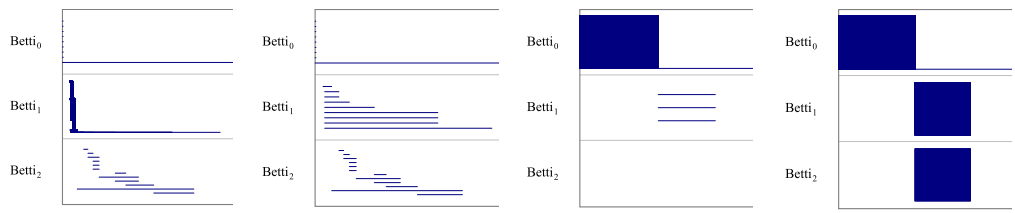
head, and spine of the user in each frame. We took the raw file from video A1 of 1,747 frames. Since there are six sensors each with x, y, z coordinates, we have in total 1747 points in $\mathbb{R}^{18}$. There are five gesture phases in the videos: rest, preparation, stroke, hold, and retraction. Indeed, there are five long bars for H_0 in 5(a) (although they overlap and do not stand out in the picture), which seems to match the five clusters of different phases. We see some long bars for H_1, which could be created due to periodic patterns in these gesture movements.

Another dataset is the Survivin protein data from [30]. There are in total 252,996 protein conformations and each conformation is considered as a point in $\mathbb{R}^{150}$. We used PCA to reduce the data dimension to 3. We ran SimBa on both datasets and show the barcodes in Figures 5(c) and 5(b). We can see that there are some long bars for H_1.

Performance Results. We provide the performance results for all datasets mentioned in Table 1, which includes cumulative size and time cost of each approach. The time is obtained by adding the time to construct the complexes and the time to compute persistence. **S.R.+GUDHI**, **S.R.+Simpers**, **B.R.+Simpers**, and **SimBa** stand for Sparse Rips plus GUDHI, Sparse Rips plus Simpers, Batch-Collapsed Rips plus Simpers, and SimBa, respectively. **Mother**, **KlBt**, **PrCi25**, **PrCi49**, **GePh**, **Sur3**, and **Sur150** stand for MotherChild model, Klein Bottle, Primary Circle in $\mathbb{R}^{25}$, Primary Circle in $\mathbb{R}^{25}$, Gesture Phase Segmentation data, Survivin protein data in $\mathbb{R}^3$, and in $\mathbb{R}^{150}$ respectively. Each dataset has size n, ambient dimension D, and intrinsic dimension d. The symbol ? means that the intrinsic dimension of the data is unknown and ∞ means that the program either ran out of memory or did not finish after a day. From the table, we can see that SimBa outperformed the other three approaches significantly. Note that for those larger cases of SimBa, the nearest-neighbor search operations (ANN) usually take the most of time and become the bottleneck. This is why **Sur150** costs much more time than **PrCi49** while its cumulative size is smaller. It would be interesting future work to make nearest neighbor search more efficient so that SimBa performs better even for such cases.

Table 2. Cumulative Size and Time Cost

Data	$\|V\|$	$\|E\|$	D	size	time(s)
Mother	23,075	–	3	100,790	13.1
KlBt	22,500	–	4	75,853	8.7
PrCi25	15,000	–	25	$5.4 \cdot 10^6$	166.9
PrCi49	15,000	–	49	$12.1 \cdot 10^6$	497
GePh	1,747	–	18	6,586	0.4
MC-Mesh	23,075	69,243	–	99,694	13.6
Facebook	4,039	88,234	–	77,048	0.8
HEP-PH	11,204	117,649	–	$74.5 \cdot 10^6$	15,916



(a) MC-Mesh (b) MC-Mesh(denoised) (c) Facebook (d) HEP-PH

Fig. 7. Persistence barcodes computed on metric graph data.

5.2 Data from General Metric Spaces

With the help of Cover tree [6] for nearest-neighbor search, we are able to build SimBa for general metric spaces. The input data is given by a symmetric distance matrix by assumption. We tested SimBa with the Cover tree approach on some of the Euclidean cases in Section 5.1. This gives us a performance comparison between ANN and Cover tree. Then, we tested with more general cases in which input data are simple undirected graphs. We consider each graph as a metric graph using the shortest-path metric. Detailed cumulative size and time cost can be found in Table 2. **MC-Mesh** is the surface mesh graph in which each edge has a positive weight for the previous **Mother** model. **Facebook** and **HEP-PH** are social network data from Facebook and the High Energy Physics collaboration network from ArXiv, both from SNAP datasets [31]. We make these graphs weighted by putting a unit weight on each edge in the experiments. Persistence barcodes are shown in Figure 7.

Performance Results. For several datasets from Euclidean space, we find that Cover tree–based implementation performs similarly to the one based on ANN: slightly worse in low dimensions and better in high dimensions. For **MC-Mesh** data, as expected, both performances and barcodes are almost the same. More general-data **Facebook** and **HEP-PH** do not have a ground truth. Since we make the simple assumption that each edge has unit weight, it is difficult to interpret the features encoded by the barcodes. The multiple $Betti_1$ and $Betti_2$ bars in the two barcodes are roughly from 0.5 to 1.0, which are due to short loops formed by one-ring neighbors. Note that in **HEP-PH** data, there are many clique sub-graphs that caused the drop in performance.

REFERENCES

[1] 2015. Simpers Software. Retrieved December 24, 2018 from http://web.cse.ohio-state.edu/ tamaldey/SimPers/Simpers.html.
[2] 2016. SimBa Software. Retrieved December 24, 2018 from http://web.cse.ohio-state.edu/ dey.8/SimBa/Simba.html.

[3] H. Adams and G. Carlsson. 2009. On the nonlinear statistics of range image patches. *SIAM J. Img. Sci.* 2, 1 (2009), 110–117. DOI: https://doi.org/10.1137/070711669

[4] U. Bauer, M. Kerber, and J. Reininghaus. 2014. *Topological Methods in Data Analysis and Visualization III: Theory, Algorithms, and Applications*. Springer International Publishing, 103–117. DOI: https://doi.org/10.1007/978-3-319-04099-8_7

[5] U. Bauer, M. Kerber, J. Reininghaus, and H. Wagner. 2017. PHAT - Persistent homology algorithms toolbox. *Journal of Symbolic Computation* 78, Supplement C (2017), 76–90. DOI: https://doi.org/10.1016/j.jsc.2016.03.008 Algorithms and Software for Computational Topology.

[6] A. Beygelzimer, S. Kakade, and J. Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning (ICML'06)*. ACM, New York, NY, 97–104. DOI: https://doi.org/10.1145/1143844.1143857

[7] J.-D. Boissonnat, T. K. Dey, and C. Maria. 2015. The compressed annotation matrix: An efficient data structure for computing persistent cohomology. *Algorithmica* 73, 3 (01 Nov 2015), 607–619. DOI: https://doi.org/10.1007/s00453-015-9999-4

[8] J.-D. Boissonnat and C. Maria. 2014. The simplex tree: An efficient data structure for general simplicial complexes. *Algorithmica* 70, 3 (1 Nov 2014), 406–427. DOI: https://doi.org/10.1007/s00453-014-9887-3

[9] M. Buchet, F. Chazal, S. Y. Oudot, and D. R. Sheehy. 2015. Efficient and robust persistent homology for measures. In *Proceedings of the 26th Annual ACM-SIAM Symposium on Discrete Algorithms*. 168–180. DOI: https://doi.org/10.1137/1.9781611973730.13

[10] D. Burghelea and T. K. Dey. 2013. Topological persistence for circle-valued maps. *Discrete Comput. Geom.* 50 (2013), 69–98.

[11] G. Carlsson. 2009. Topology and data. *Bull. Amer. Math. Soc.* 46 (2009), 255–308.

[12] G. Carlsson and V. de Silva. 2010. Zigzag persistence. *Foundations of Computational Mathematics* 10, 4 (2010), 367–405.

[13] G. Carlsson and A. Zomorodian. 2009. The theory of multidimensional persistence. *Discrete & Computational Geometry* 42, 1 (2009), 71–93. DOI: https://doi.org/doi:10.1007/s00454-009-9176-0

[14] N. J. Cavanna, M. Jahanseir, and D. R. Sheehy. 2015. A geometric perspective on sparse filtrations. In *Canadian Conference on Computational Geometru (CCCG'15)*. http://dblp.uni-trier.de/db/conf/cccg/cccg2015.html#CavannaJS15

[15] F. Chazal, D. Cohen-Steiner, M. Glisse, L. J. Guibas, and S. Y. Oudot. 2009. Proximity of persistence modules and their diagrams. In *Proceedings of the 25th Annual Symposium on Computational Geometry (SCG'09)*. New York, NY, 237–246. DOI: https://doi.org/10.1145/1542362.1542407

[16] F. Chazal, D. Cohen-Steiner, L. Guibas, F. Mémoli, and S. Y. Oudot. 2009. Gromov-Hausdorff stable signatures for shapes using persistence. In *Proceedings of SGP*.

[17] F. Chazal, V. de Silva, M. Glisse, and S. Oudot. 2012. The structure and stability of persistence modules. *CoRR* abs/1207.3674 (2012).

[18] F. Chazal, L. J. Guibas, S. Y. Oudot, and P. Skraba. 2011. Persistence-based clustering in Riemannian manifolds. In *Proceedings of the 27th Annual ACM Symposium on Computational Geometry*.ACM, 97–106.

[19] C. Chen and M. Kerber. 2011. Persistent homology computation with a twist. In *Proceedings of the 27th European Workshop on Computational Geometry*.

[20] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. 2007. Stability of persistence diagrams. *Discrete & Computational Geometry* 37, 1 (2007), 103–120.

[21] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. 2009. Extending persistence using Poincaré and Lefschetz duality. *Foundations of Computational Mathematics* 9, 1 (2009), 79–103.

[22] D. Cohen-Steiner, H. Edelsbrunner, and D. Morozov. 2006. Vines and vineyards by updating persistence in linear time. In *Proceedings of the 22nd Annual Symposium on Computational Geometry*. ACM, 119–126.

[23] V. de Silva, D. Morozov, and M. Vejdemo-Johansson. 2011. Persistent cohomology and circular coordinates. *Discrete Comput. Geom.* 4 (2011), 737–759.

[24] T. K. Dey, F. Fan, and Y. Wang. 2014. Computing topological persistence for simplicial maps. In *Proceedings of the 30th Annual Symposium on Computational Geometry (SOCG'14)*. Article 345, 10 pages. DOI: https://doi.org/10.1145/2582112.2582165

[25] T. K. Dey, K. Li, C. Luo, P. Ranjan, I. Safa, and Y. Wang. 2010. Persistent heat signature for pose-oblivious matching of incomplete models. *Comput. Graph. Forum (special issue from Sympos. Geom. Process.)* 29, 5 (2010), 1545–1554.

[26] Dmitriy Morozov. 2012. Dionysus Software. Retrieved December 24, 2018 from *http://mrzv.org/software/dionysus/*.

[27] H. Edelsbrunner and J. Harer. 2010. *Computational Topology - An Introduction*. American Mathematical Society. I–XII, 1–241 pages.

[28] H. Edelsbrunner, D. Letscher, and A. Zomorodian. 2002. Topological persistence and simplification. *Discrete Comput. Geom.* 28 (2002), 511–533.

[29] P. Frosini. 1990. A distance for similarity classes of submanifolds of a Euclidean space. *Bulletin of the Australian Mathematical Society* 42, 3 (1990), 407–416.
[30] W. Harvey, I.-H. Park, O. RAijbel, V. Pascucci, P.-T. Bremer, C. Li, and Y. Wang. 2014. A collaborative visual analytics suite for protein folding research. *Journal of Molecular Graphics and Modelling* 53 (2014), 59–71. DOI : https://doi.org/10.1016/j.jmgm.2014.06.003
[31] J. Leskovec and A. Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. *R*etrieved December 24, 2018 from http://snap.stanford.edu/data.
[32] M. Lichman. 2013. UCI Machine Learning Repository. Retrieved December 24, 2018 from http://archive.ics.uci.edu/ml.
[33] R. C. B. Madeo, S. M. Peres, and C. A. de M. Lima. 2016. Gesture phase segmentation using support vector machines. *Expert Systems with Applications* 56 (2016), 100–115. DOI : https://doi.org/10.1016/j.eswa.2016.02.021
[34] R. C. B. Madeo, P. K. Wagner, and S. M. Peres. 2014. Gesture Phase Segmentation Data Set. Retrieved December 24, 2018 from http://archive.ics.uci.edu/ml/datasets/Gesture+Phase+Segmentation.
[35] D. M. Mount and S. Arya. 2010. ANN: A Library for Approximate Nearest Neighbor Searching. Retrieved December 24, 2018 from https://www.cs.umd.edu/~mount/ANN/.
[36] James R. Munkres. 1993. *Elements of Algebraic Topology.* Addison-Wesley.
[37] J. Reininghaus, S. Huber, U. Bauer, and R. Kwitt. 2015. A stable multi-scale kernel for topological machine learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'15).* IEEE, 4741–4748.
[38] V. Robins. 1999. Towards computing homology from finite approximations. *Topology Proceedings* 24, 1 (1999), 503–532.
[39] D. R. Sheehy. 2012. Linear-size approximations to the vietoris-rips filtration. In *Proceedings of the 28th Annual Symposium on Computational Geometry (SoCG'12).* 239–248. DOI : https://doi.org/10.1145/2261250.2261286
[40] G. Singh, F. Mémoli, T. Ishkhanov, G. Sapiro, G. Carlsson, and D. L. Ringach. 2008. Topological analysis of population activity in visual cortex. *Journal of Vision* 8, 8 (2008), 11.
[41] The GUDHI Project. 2015. *GUDHI User and Reference Manual.* GUDHI Editorial Board. Retrieved December 24, 2018 from http://gudhi.gforge.inria.fr/doc/latest/.
[42] A. Zomorodian and G. Carlsson. 2005. Computing persistent homology. *Discrete Comput. Geom.* 33, 2 (2005), 249–274.

Received April 2017; revised September 2018; accepted September 2018